

RAPPORT CRYPTOGRAPHIE APPLIQUÉE

SUJET 3 : RÉSEAUX EUCLIDIENS ET PYTHON

lien git

https://gitlab.esiea.fr/bouvierdacher/cryptographie_cryptis

Introduction

Le but du sujet est de coder le logiciel cryptis en python. Cryptis est un jeu permettant de comprendre la cryptographie grâce aux réseaux euclidiens. Elle se base sur le principe de clef publique et clef privée. La cryptographie à base de réseaux euclidiens repose sur le problème du plus court vecteur, un problème NP-complet.

Principe de cette cryptographie

La cryptographie avec les réseaux euclidiens fonctionnent grâce à la création de clef de taille N où chaque case représente un vecteur dans un plan à N dimension avec N plan où chaque plan correspond à une rotation de clef possible, si l'on fusionne chaque plan, cela donnera toutes points atteignables par la clef. La clef privée sera une clef qui sera quasi orthogonale au plan, c'est à dire que toutes les cases de la clef sera proche du centre 0 sauf une qui elle sera plus grande. Le clef publique sera un multiple de la clef privée crée de telle sorte que la clef soit le moins orthogonal au plan, c'est à dire que la grande majorité de ces chiffres soit éloigné de 0.

Dictionnaire

Pour pouvoir chiffrer nos messages, il faut créer le dictionnaire permettant de convertir les caractères en trits, qui seront ensuite utilisable par notre algorithme de chiffrement et déchiffrement. J'ai utilisé les 128 premiers caractères du tableau ASCII, ce qui correspond à des caractères de taille 5, car $3^4 = 81$ et $3^5 = 243$ comme $81 < 128 < 243$, pour avoir les 128 premiers caractères ASCII possible, il faut convertir chaque caractère en 5 trits.

Clef privée

La création de la clef privée repose sur un principe simple, il faut que toute les cases sauf une soit proche de 0, la dernière devra être éloignée de 0. La première case du tableau sera égal à la taille de la clef, pour les autres cases, on utilise la fonction $f(x) = x/4$ pour x partant de 0 jusqu'à N-1 où N est la taille de ma clef, puis pour crée des clefs différentes, on applique à chaque caractère de la clef la fonction $f(x) = \pm x/8$ donc la fonction pour clef la clef privée et $f(x) = x/4 \pm x/8$ pour les N-1 caractère et $f(x) = N \pm x/8$ pour le dernier, suite à ça, il faut mélanger la clef pour obtenir une clef complètement aléatoire.

Clef publique

La création de la publique demande d'avoir la clef privée. En effet, la clef publique est un multiple de la clef privée. Une fois que l'on a la clef privée, la clef publique sera créer de manière aléatoire, comme la clef publique est un multiple de la clef privée, on ajoutera plusieurs fois la clef privée, sous ses N possibilité, ce qui nous donnera une clef publique. Pour qu'elle soit efficiente, il faut que la majorité des cases soit éloignées de 0. Pour ce fait, il faut que la majorité des possibilités de la clef privée soit ajouté pour former une bonne clef publique.

Chiffrement

Pour pouvoir chiffrer notre message, il faut ajouter plusieurs fois notre clef publique, sous ses différentes possibilités, au message.

Déchiffrement

Il y a plusieurs algorithmes permettant de déchiffrer un message chiffré. Celui utiliser repose sur l'algorithme de Babai (cf Source). C'est un algorithme d'approximation du plus proche vecteur. C'est à dire qu'avec notre clef privée, nous essayer de nous approcher le plus proche possible de notre message chiffré avec notre clef publique. Il est approximatif, car comme notre message est suite de 1, 0, -1, notre algorithme lui tentera d'arrivé à un message contenant que des 0 car les 1 et -1 ne font pas partie de son champ des possibles. Ce qui donnera un résultat avec des chiffres flottants, qui peut dans certains cas être problématique si la clef privée est une mauvaise base. A contrario, si la base est bonne, l'algorithme nous donnera le résultat optimal.

C'est un algorithme de résolution de système, pour trouver quel poids à chaque possibilité de clef.

Par exemple :

Admettons la clef est de taille 3,

clef_privée = [1, 7, -1],

clef_publique = [16, -4, -12],

message = V = [-1, 1, 0, 0, 1]

message_chiffre = [-9, 29, -20, 44, -23, -20]

on crée une matrice avec toute les possibilités de clef

clef = [[1, 7, -1],
[-1, 1, 7],
[7, -1, 1]]

on applique la transposée de la matrice pour avoir les poids de chaque clef par ligne et non par colonne

clef = [[1, -1, 7],
[7, 1, -1],
[-1, 7, 1]]

et on résout le système

$$1*x - 1*y + 7*z = -9$$

$$7*x + 1*y - 1*z = 29$$

$$-1*x + 7*y + 1*z = -20$$

une fois résolu le système donne :

$$x = 107/26 = 4,115$$

$$y = -51/26 = -1,961$$

$$z = -28/13 = -2,153$$

Si l'on arrondi, il faut appliquer 4 fois la clef [1, 7, -1], -2 fois la clef [-1, 1, 7] et 2 fois la clef [7, -1, 1].

Si on applique le résultat trouvé, on retrouve le message initial pour les 3 premiers caractères. Pour retrouver tout le message, il faut continuer l'algorithme sur le reste du message.

Quel résultat donnera une clef privée avec une mauvaise base ?

Le résultat obtenu par l'algorithme décrit au-dessus donnera un résultat proche de la réalité. Avec une mauvaise base, on reste proche de la réalité, mais le résultat devient ambigu. En effet, une fois le résultat obtenu, on applique un arrondi au plus proche, c'est à dire que 1,2 devient 1 et 1,8 devient 2. Si le résultat que l'on obtient est 1,48, faut-il arrondir au supérieur ou à l'inférieur. Une clef avec une mauvaise base donnera à chaque fois des réponses ambiguës, ce qui modifiera les messages déchiffrés.

Par exemple :

Soit une clef de taille 10.

clef_privée = [1, 0, 0, 1, 4, 3, -2, 0, 6, 4],

clef_publique = [-13, -58, -49, -23, -3, 3, -27, -58, -23, 13],

message = Victoire

Quelle que soit la manière dont le message est chiffré, le résultat du message déchiffré après le chiffrement sera toujours « ctoire ». Le Vi de Victoire a donc disparu. Quand on regarde le nombre de chiffres entre x_4 et x_6 , on constate que pour le déchiffrement de Vi, il y en a 3 alors que pour les autres, ils sont inférieurs à 2. Le résultat est ambigu et est donc faux pour ce cas là.

Déroulement de la cryptographie

Tout d'abord, il faut créer la clef privée et publique. Ensuite, il nous faut un message à convertir, tout d'abord, nous allons le convertir en trits puis le chiffrer. Une fois chiffré, il faut le déchiffrer et le reconvertir en caractère. Et on a le message de départ.

Stabilité de l'algorithme de déchiffrement

Maintenant que l'on a tout ce qu'il faut pour pouvoir chiffrer et déchiffrer nos messages, il faut vérifier la robustesse de notre algorithme de déchiffrement et tester la clef publique sur cet algorithme. C'est là que l'on constate que l'algorithme de déchiffrement fonctionne aussi avec la clef publique, ce qui est un gros problème, car cela rend obsolète le chiffrement avec les réseaux euclidiens. La clef publique est même plus efficace pour déchiffrer les messages si la clef privée est une clef avec une mauvaise base. On peut donc se poser les questions suivantes, est-ce que toutes les clefs publiques possibles créées à partir d'une clef privée peuvent déchiffrer les messages ? Si oui, alors les réseaux euclidiens deviennent compliqués à utiliser pour pouvoir être utilisés dans la cryptographie. Si non, Quels types de clef publique ne permettent pas de déchiffrer les messages ? Comme on l'a vu précédemment, certaines clefs privées avec une mauvaise base n'arrivaient pas à déchiffrer entièrement les messages. Il faudrait pouvoir créer de manière efficace des clefs de mauvaises bases. La question est de savoir comment pouvoir créer des clefs publiques de mauvaises bases.

Source

1. Algorithme de Babai

<https://staff.emu.edu.tr/alexanderchefranov/Documents/CMSE491/CMSE491%20Fall2020/Hoffstein2015%20Introduction%20to%20Mathematical%20Cryptography%20403-407%20Babai.pdf>